AI ACADEMY · LEVEL 1 · GRADE 6 · WEEK 12 · STUDENT LESSON

Debug the Cause

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Helpful Classroom Sorter

THE CONNECTED WEEK

Keep the failure. Choose a revealing test.

Find the first divergence, compare fault predictions, repair a maze route and debug a new ticket boundary.

Keep one habit: State the expectation → preserve the observation → predict a test that separates explanations.

MAIN LESSON · SECTION 1

Which test would reveal the cause?

> Keep the specification, observed trace and proposed explanation distinguishable.

Start with a wrong report

A final number is wrong. Before changing it, compare the expected and observed intermediate states. Then choose a new test whose predictions differ under the suspected explanations.

Retrieve the loop's final test

Start at 3. While counter is below 7, add 2. List the values at every test and state when the body stops running.

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

By the end, you can...

- I can compare **expected** and observed **trace**s to locate the first difference.
- I can choose a test that distinguishes competing explanations of a fault.
- I can repair a stated instruction and rerun both the failing case and earlier passing cases.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

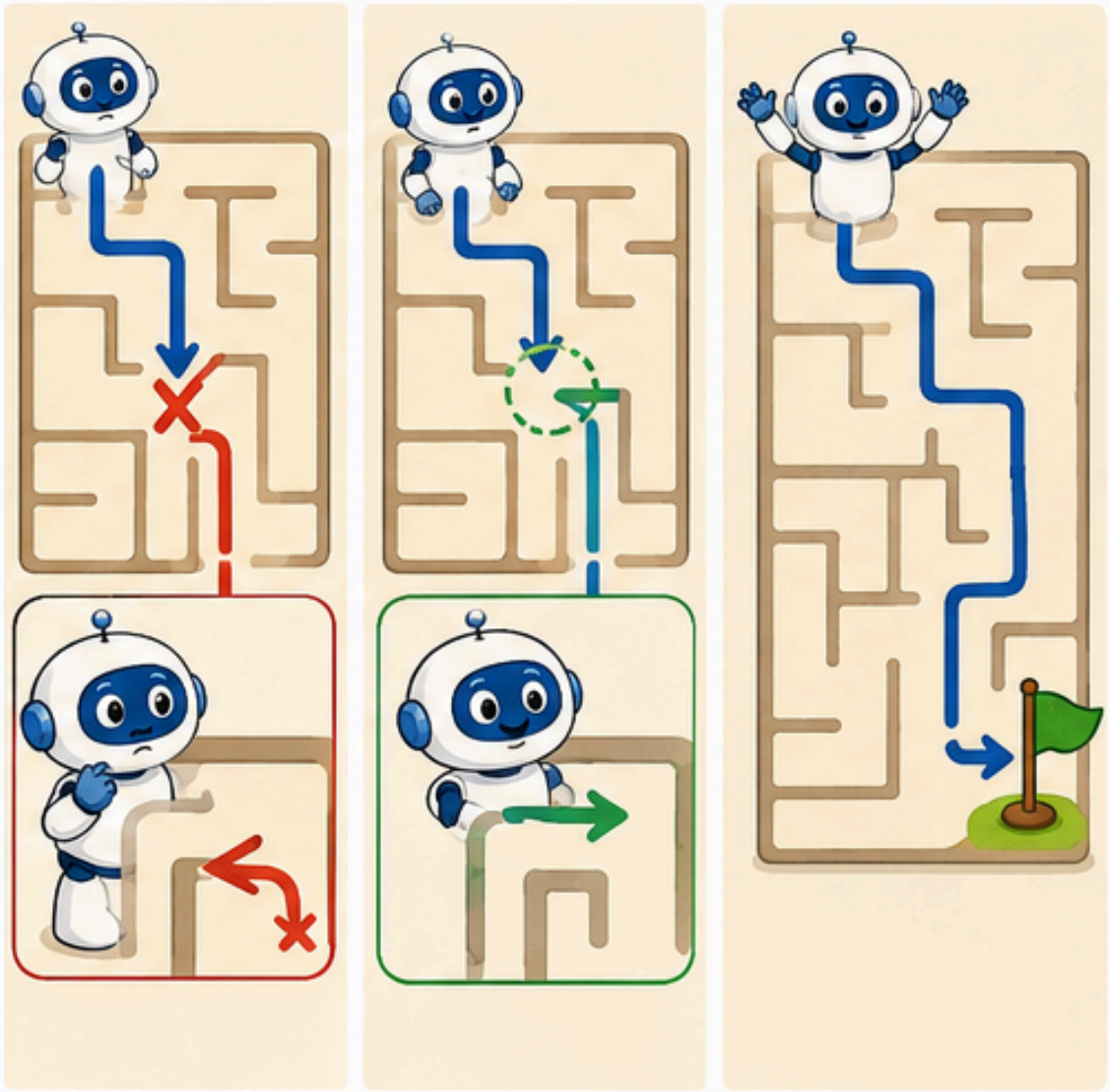
Remember before reading

Without opening the old lesson, explain one key idea from Repeating Efficiently. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Algorithms as Recipes. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Sound Becomes a Signal. Give an example and a mistake that the idea helps you avoid.

Debugging finds the broken assumption or step before changing the solution.



LEARNING TARGET Starting at square 0, the target is square 3. Program: repeat twice {MOVE 1}; STOP. A teammate changes the target to 2 and calls the bug fixed. Repair the program and give two tests.

Learner name:

Readiness for the connected explanation

This week’s end goal

By the end, I can choose a test that separates two possible causes of an error.

MAIN LESSON · SECTION 2

Locate the first divergence before repairing

> Use the stated arithmetic procedure to calculate an expectation before inspecting the faulty trace.

Read the specification

A paper points machine must add 3 to its starting score, double the updated score, then subtract 2 and report. From 4 the **expected** values are 4, 7, 14 and 12. These instructions are the **specification**.

Calculate the **expected result** from them before looking at the machine **output**.

Inspect the recorded faulty run

An observed run instead records 4, 7, 11 and 9. The final result is wrong, but changing the report from 9 to 12 would only hide this particular failure.

Compare the intermediate values to find where the behaviour first stopped matching the instructions.

Distinguish the first difference from its later effects

The first difference is after the doubling instruction: **expected** 14, observed 11. The final subtraction is consistent with the observed 11, since 11 minus 2 is 9.

That later difference can be a consequence of the earlier fault rather than a second faulty subtraction. A **trace** narrows the investigation; it does not automatically reveal every cause.

Explain the earliest mismatch

- Mark the first row where expected and observed values differ. Why can the final subtraction follow its instruction correctly while its answer is still different from the expected result?

▼ Earlier wording, alternatives and picture transcripts

PICTURE STORY

Look Closely Before You Explain

Pictures give clues; **evidence** tells us what the clues mean.



MYSTERY A paper robot hits a wall because one turn instruction assumes it faces north when it **actual**ly faces east.

What do you notice?

Name three visible details without explaining them yet.

Circle the detail most connected to the system's decision.

Name one detail that may be decoration or distraction.

Make a first **prediction** and state the clue that supports it.

My **observation**s, **prediction**, and **evidence**:

GOOD THINKING A **prediction** can change when stronger **evidence** appears.

A closer explanation

Keep an expected column and an observed column

> A reproducible bug record includes the starting input and exact instruction version.

Compare the trace row by row

Locate the first mismatch

RECORDED CHECKPOINT	COMPARE THE VALUES	WHAT THE EVIDENCE TELLS US
Start	Expected 4 observed 4	Values agree
After add 3	Expected 7 observed 7	Values agree
After double	Expected 14 observed 11	First difference
After subtract 2	Expected 12 observed 9	Difference continues

Each row compares the same checkpoint in two traces. Arrows lead from the checkpoint to its comparison and interpretation; they do not represent arithmetic between columns.

Retain the evidence needed for reproduction

Record the **input**, the exact instruction version, **expected** values, observed values and first mismatch.

This makes the failure reproducible: someone else can follow the same procedure with the same starting value and check your **observation**.

Keep the specification stable during a test

Do not rewrite the **expected** column to match the **observed result**. If the **specification** itself seems ambiguous, clarify it first and record that change openly.

A test can judge an implementation only against a stated expectation.

Explain one deciding row

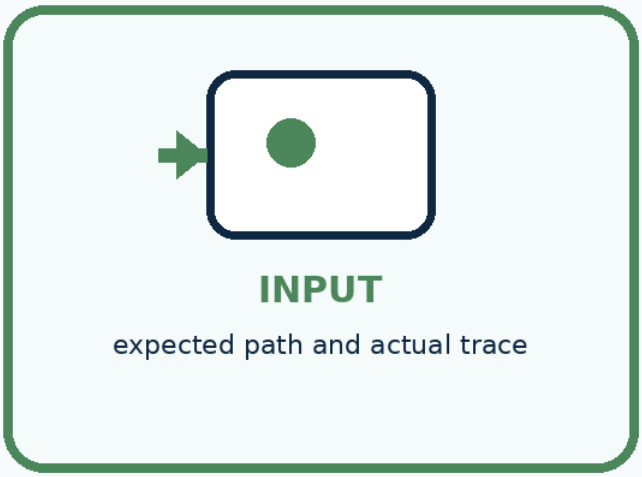
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

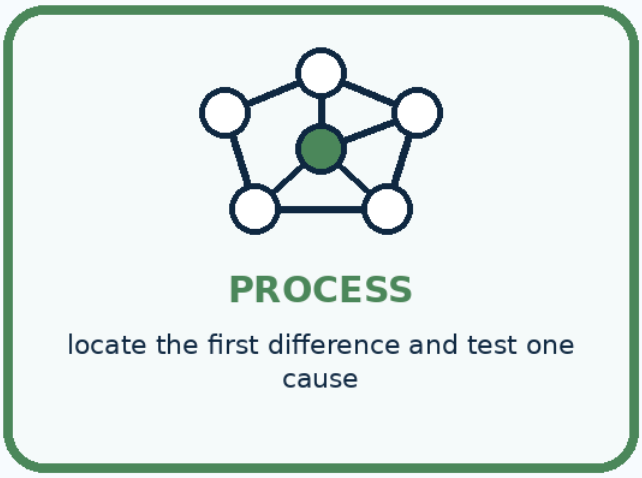
VISUAL **MECHANISM**

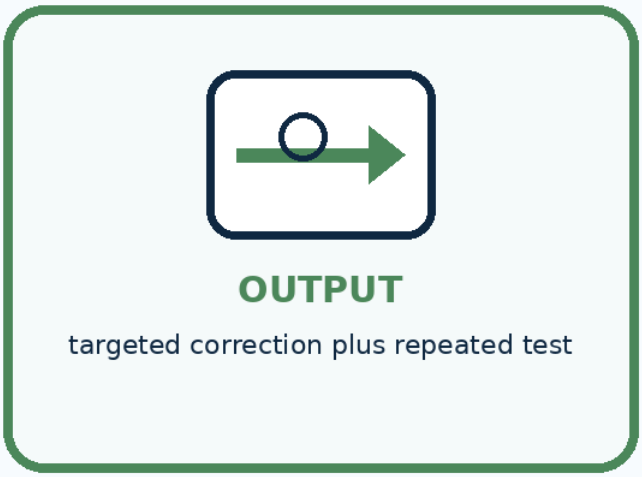
Input → Process → Output

Follow the information instead of imagining magic inside the machine.



</details>





INPUT expected path and **actual trace**

PROCESS locate the first difference and test one cause

OUTPUT targeted correction plus repeated test

CORE IDEA Debugging finds the broken assumption or step before changing the solution.

UNDER THE HOOD Debugging is causal diagnosis. The first observable divergence narrows the faulty assumption, data, state, condition, or step.

Changing multiple things at once destroys **evidence** about which change repaired the behavior.

Draw a fresh example with all three stages:

A closer explanation

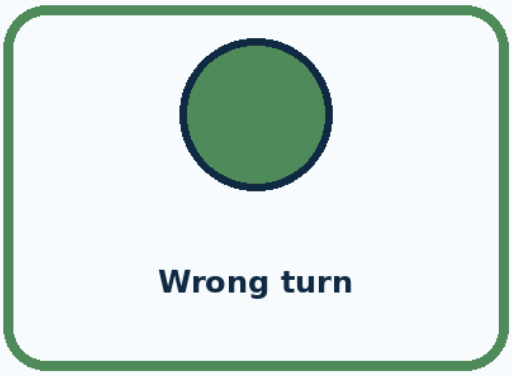
Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare symptoms, traces and possible causes

> Choose evidence that separates explanations rather than patching only the visible result.

Pictures paired with their evidence



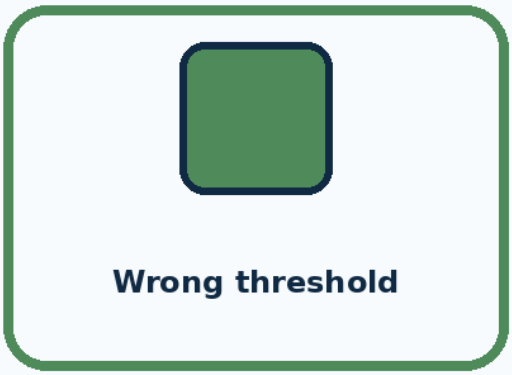
The case to investigate

CASE 1 — Wrong turn Clue: **first divergence** at step 4. Meaning: inspect orientation assumption.



The case to investigate

CASE 2 — Missing card Clue: **input** absent before step 1. Meaning: data/setup cause.



The case to investigate

CASE 3 — Wrong threshold Clue: branch changes at 10. Meaning: condition cause.



The case to investigate

CASE 4 — Random rewrite Clue: many steps changed. Meaning: weak debugging **evidence**.

Discuss the confusing pair

- Which two pictures are easiest to confuse? What **evidence** separates them?

Keep the limit in view

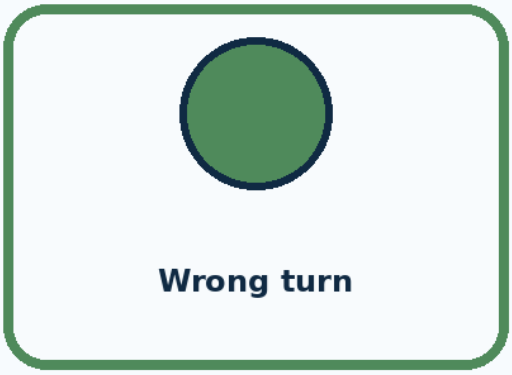
BOUNDARY A successful new run does not prove the diagnosis if several things changed together.

▼ Earlier wording, alternatives and picture transcripts

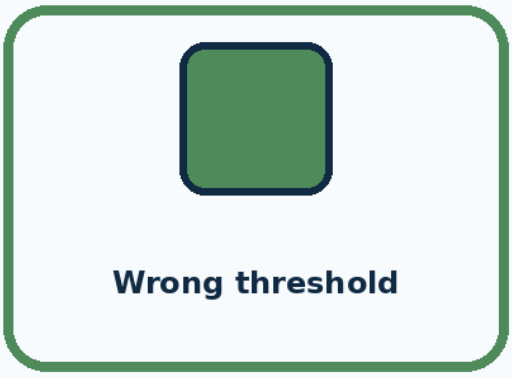
PICTURE GALLERY

Four Cases, Four Clues

Use the visual cue, then read the **evidence** carefully.









CASE 1 — Wrong turn Clue: **first divergence** at step 4. Meaning: inspect orientation assumption.

CASE 2 — Missing card Clue: **input** absent before step 1. Meaning: data/setup cause.

CASE 3 — Wrong threshold Clue: branch changes at 10. Meaning: condition cause.

CASE 4 — Random rewrite Clue: many steps changed. Meaning: weak debugging **evidence**.

Change one thing in the pictured case

Change: Use **input** 6 again instead of 2.

Prediction: Both hypotheses predict 12; the test remains inconclusive.

Reason: A revealing test requires different predicted **output**s, not merely another recorded run.

MAIN LESSON · SECTION 6

Choose an input that separates the hypotheses

> A new test is revealing when the alternatives predict different results before it runs.

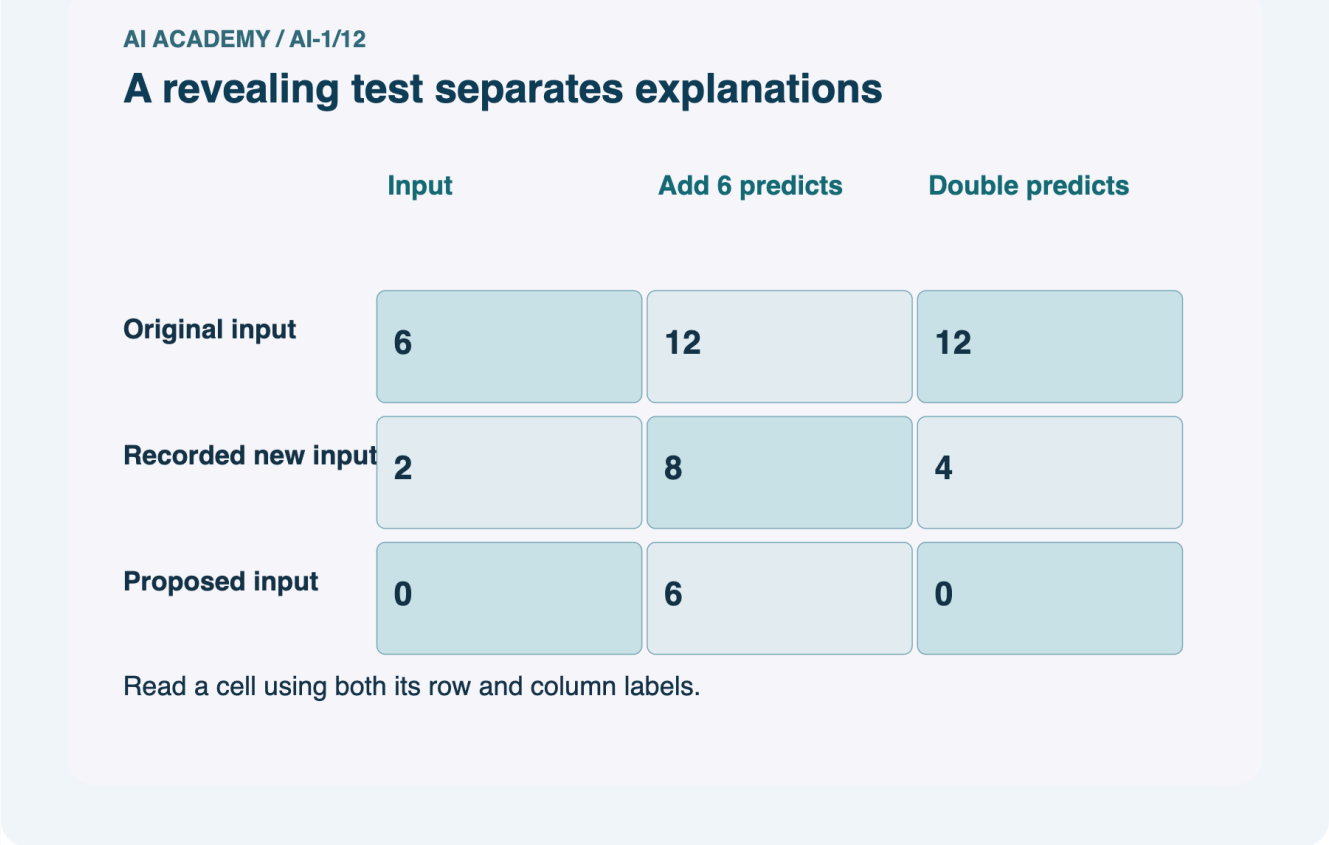
The two hypotheses and first observation

A fictional paper calculator should double the current value, but a copied instruction behaves strangely.

After **input** 6, the faulty step reports 12. Hypothesis A: add 6. Hypothesis B: double.

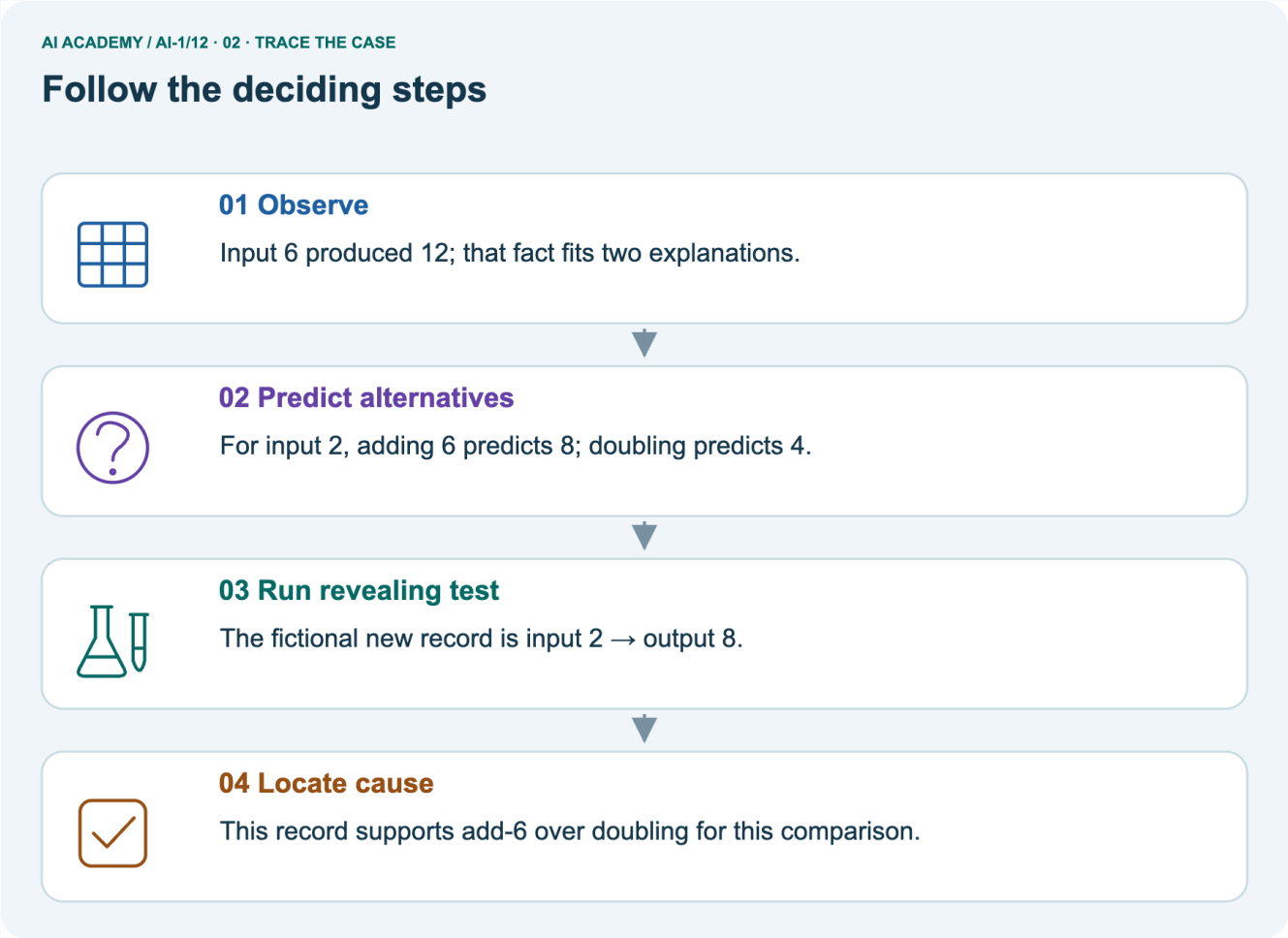
Both hypotheses fit this first **observation**. A new **input** may be chosen.

Read the contrasting predictions



Follow the labelled picture with the matching explanation. Open the vector diagram

Trace the fictional revealing test



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain the evidence and its remaining limit

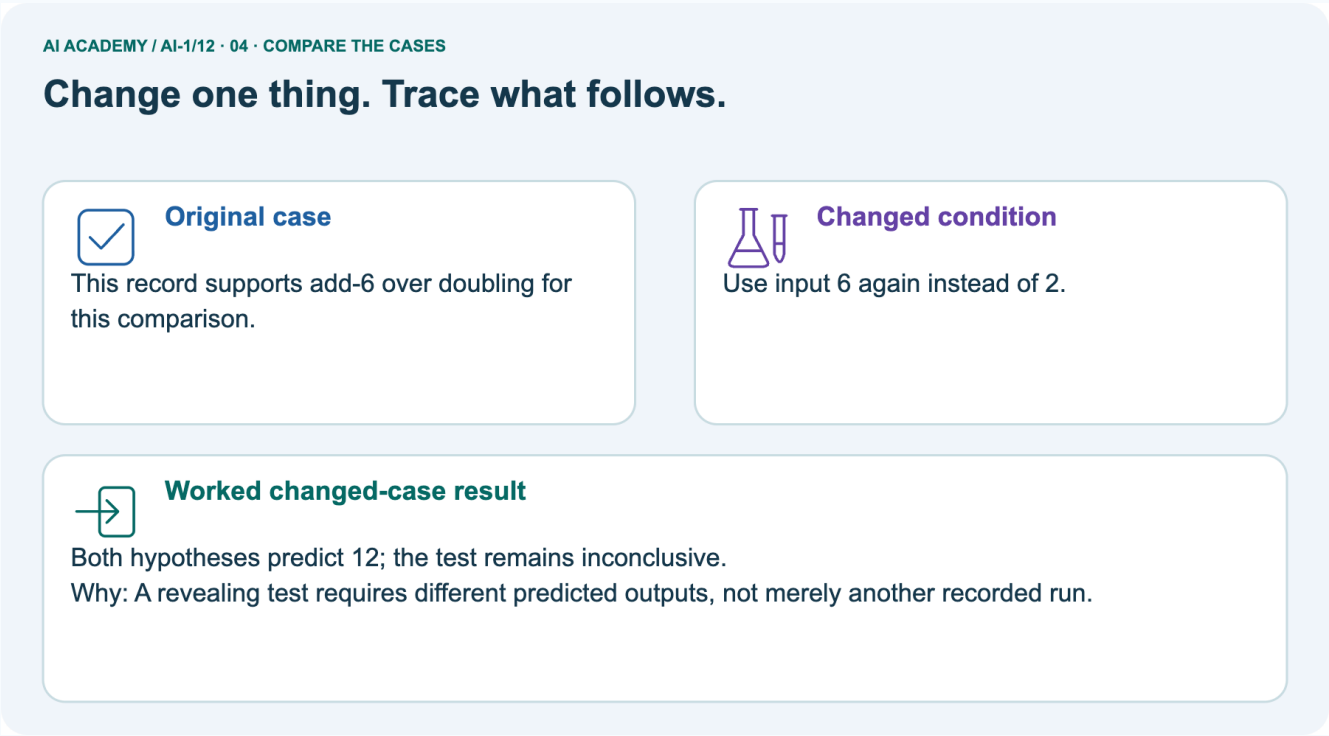
Repeating **input** 6 would not separate the two hypotheses because both again predict 12. **Input** 2 makes their **prediction**s different before the test is run.

The second record supports investigating an added constant, but inspecting the instruction is still needed to confirm the implementation.

Predict a repeated input of 6

- Before checking, use input 6 again. Predict the output if the step adds 6, and if it doubles its input. Would those predictions separate the two explanations? Keep predicted values separate from a new observed record.

▼ Check the inconclusive comparison



Follow the labelled picture with the matching explanation. Open the vector diagram

Do not claim a unique hidden implementation

Two toy hypotheses do not exhaust every possible bug; a matching **output** alone cannot identify arbitrary hidden code.

Propose input 0 without inventing a result

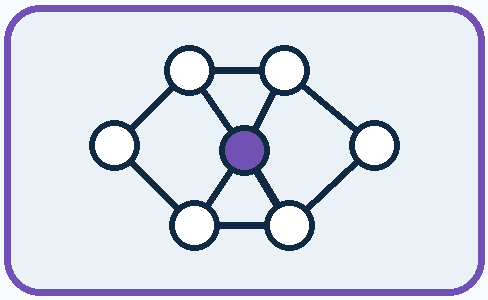
- Read the labelled picture: Propose **input** 0 and give both predicted **output**s without inventing an **observed result**.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

WORKED EXAMPLE

Watch the Reasoning, Not Just the Answer

Every step connects a claim to observable **evidence**.



CASE A paper robot hits a wall because one turn instruction assumes it faces north when it **actual**ly faces east.

STEP 1 — OBSERVE **expected** path and **actual trace**

STEP 2 — **TRACE** locate the first difference and test one cause

STEP 3 — CONCLUDE targeted correction plus repeated test

MODEL REASONING **Trace expected** and **actual** positions step by step. The first mismatch reveals the earliest plausible cause.

Change that instruction or assumption only, then rerun the identical maze to show whether the cause was correct.

DO NOT OVERCLAIM A successful new run does not prove the diagnosis if several things changed together.

What new **evidence** could change this answer?
</details>

A closer explanation

For the faulty doubling step, two explanations fit the observed change from 7 to 11. Perhaps the step adds 4. Perhaps it adds the original starting score, which was 4.

Repeating the same start 4 cannot separate those explanations because they make the same **prediction**.

Start score	Expected final result	If step adds 4 or original start
4	12	9 under either explanation
2	8	7 if add 4 or 5 if add original 2
5	14	10 if add 4 or 11 if add original 5

Starting at 2 produces 5 after the first addition. The intended double gives 10 and then reports 8. Adding 4 would give 9 and report 7. Adding the original 2 would give 7 and report 5.

The different **prediction**s make this a revealing test.

If the observed final result is 7, it supports add 4 over add original among these two explanations. Other faults could still give 7, so inspect the **actual** instruction or more checkpoints before claiming a unique cause.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debug the Cause

The facts we will use

AI ACADEMY / AI-1/12

A revealing test separates explanations

	Input	Add 6 predicts	Double predicts
Original input	6	12	12
Recorded new input	2	8	4
Proposed input	0	6	0

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-1/12

Debug the Cause

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Observe

Input 6 produced 12; that fact fits two explanations.



2. Predict alternatives

For input 2, adding 6 predicts 8; doubling predicts 4.



3. Run revealing test

The fictional new record is input 2 -> output 8.



4. Locate cause

This record supports add-6 over doubling for this comparison.

CHANGE ONE THING

Use input 6 again instead of 2.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Observe	Input 6 produced 12; that fact fits two explanations.
2. Predict alternatives	For input 2, adding 6 predicts 8; doubling predicts 4.
3. Run revealing test	The fictional new record is input 2 → output 8.
4. Locate cause	This record supports add-6 over doubling for this comparison.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

What this example does not establish

Change one thing: Use **input** 6 again instead of 2. Predict the result before checking the explanation. What remains the same?

Make each test check a stated risk

> Retain both the failure evidence and the limits of a passing test set.

Choose arithmetic, equality and loop tests deliberately

For an arithmetic step, choose **input**s that produce different **prediction**s under plausible faults. For a condition, test its equality case as well as values on each side.

For a loop, include a false first test and a run that crosses its stopping boundary. A test list is stronger when you can explain what each case checks.

Write a reproducible bug report

A useful bug report names the version and **input**, says what should happen, records what did happen, and identifies the first known difference. Include the repair and retest **evidence** once available.

Passing a finite collection of tests increases confidence in those behaviours; it does not prove every possible **input** correct.

Design a regression test

- CLAIM A Debugging means rewriting everything that looks suspicious.
- CLAIM B The last visible error must be the original cause.

STRONG ANSWER FRAME I claim _____ because the **evidence** shows _____. The **process** is _____. One limit is _____.

> **A repaired below 7 loop is checked only from start 1. Name a regression test that would reveal an accidental change from below 7 to at most 7, give both expected outcomes with add 2, and explain why the test is revealing**

▼ Earlier wording, alternatives and picture transcripts

COMPARE

Same Result, Different Reason

A comparison helps reveal the true concept boundary.

Case	Evidence	What it shows
Wrong turn	first divergence at step 4	inspect orientation assumption
Missing card	input absent before step 1	data/setup cause
Wrong threshold	branch changes at 10	condition cause
Random rewrite	many steps changed	weak debugging evidence

Repair two tempting claims

Repair:

Repair:

A closer explanation

MAIN LESSON · SECTION 8

Practise: repair a confirmed instruction and retest

> Keep the failing input, rerun earlier cases and change one suspected instruction at a time.

Apply the repair only after inspection confirms it

Suppose inspection confirms that the second instruction says add 4. Replace that instruction with double the current score. Keep the original failing test, then rerun other **input**s.

From starts 4, 2 and 5, the corrected final results should be 12, 8 and 14.

> **A copied machine reports 9 from start 4 and 7 from start 2. Which of the two proposed explanations is consistent with both results? State what to inspect, the repair if confirmed, and the expected results to record after the repair**

Keep inputs fixed while comparing the repair

Change the suspected instruction while keeping the test **input**s fixed. If you change several instructions and **input**s together, it is harder to tell which change mattered.

Rerun previously passing cases as **regression test**s, because a repair can accidentally break other behaviour.

Use W2 to distinguish the suspected faults

Use workbook W2 for the connected example “Debugging with revealing tests”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your **trace** to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Challenge “the last wrong number locates the fault”

- Use the worked case for Debugging with revealing tests. Proposed explanation: “The last wrong number locates the fault”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured input-0 predictions

Propose **input** 0 and give both predicted **output**s without inventing an **observed result**.

> **My explanation using a pictured step and the given facts**

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

GUIDED PRACTICE

Try It, Check It, Explain It

Use support now so you can succeed independently later.



Explain how the picture story demonstrates this idea: Debugging finds the broken assumption or step before changing the solution.

Use a fresh example to explain bug.

Trace a new **input** → **process** → **output** example.

Find and repair the error in: The last visible error must be the original cause.

Invent one edge case and name the safe response.

PARTNER CHECK Point to the **evidence**. Do not accept an answer that only repeats the vocabulary word.

A closer explanation

Practise with less help: Choose between two explanations

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Investigate a maze failure with a stable target

> Use exact movement rules and stop at the first failed requirement.

Map, commands and required result

Use a fictional 3-by-3 maze with rows 0–2 top to bottom and columns 0–2 left to right. Start at (0,0), target (2,2), blocked cell (1,1). R adds 1 to column; D adds 1 to row. A move outside the grid or into a blocked cell fails before changing position, and the trace stops. After a completed recipe, report the final position; success requires the target. Faulty recipe: R,D,D,R.

Predict, locate, repair and retest

1. Predict the first failed move and trace the faulty recipe on paper. Repair the route without moving the target or erasing the blocked cell. Before tracing the repair, write its expected positions. Retest the repaired route on this maze and on a version with no blocked cell, recording both map versions. Keep an actual paper trace separate from a prediction.

2. Keep a normal case and an edge case clearly separated.
3. Record what happened—even when it does not match your **prediction**.
4. Explain the result, one limitation, and the next useful test.

Keep the first failure and regression evidence

Case	Prediction	Observed result	What it means
Normal			
Edge			

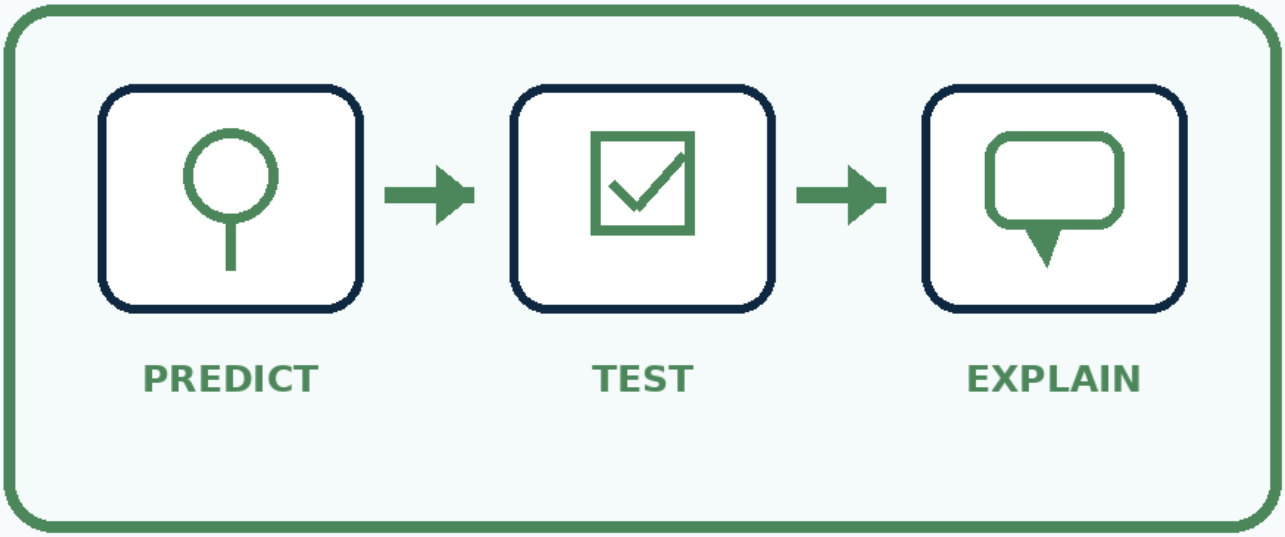
Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I **actual**ly observe, and what should change? Keep “not run” if you only **trace**d the procedure.

▼ Earlier wording, alternatives and picture transcripts

EXPLORER LAB

Maze Cause Detective

Predict first, test honestly, and explain what the **evidence** means.



State your **prediction** before the result is known.

Complete the task: Repair a maze algorithm and explain the exact failure.

INDEPENDENCE After support is removed, repeat the proof with a fresh case.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: debug the new ticket boundary

> Choose starting inputs that reach below, at and above the comparison after its update.

The new specification and copied comparison

A paper ticket rule must add 1 to the **input** count. If the updated count is at least 5, report Group; otherwise report Single. A copied version says above 5.

Choose the smallest three consecutive nonnegative starting counts that test just below, exactly at and just above the comparison boundary. Predict both versions, identify the revealing start and state the repair and retests.

My independent explanation, trace and limits

Keep one first attempt

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Debug a new ticket rule

Week 12 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

A sorting rule says blue goes left. A blue card goes right. The reader correctly reports blue, but the direction rule says blue goes right. Which part should you fix first?

My answer, with a **trace**, calculation or supporting **evidence**

> **Write or draw your reasoning and evidence.**

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and **expected result**

> **Write or draw your reasoning and evidence.**

Connect to your project

When the sorter fails, log the first wrong stage and change only the responsible rule, **input**, or label.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

The requirement is to move exactly three squares forward from any valid starting square, then stop. Starting at square 0, repeat twice {MOVE 1}; STOP ends at 2. A teammate reduces the required distance to two squares.

Repair the program without changing the requirement; test starts 0 and 2.

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Keep competing explanations in your sorter’s bug log

> Write expected traces before changing instructions.

This week’s project decision

THIS WEEK'S CHECKPOINT When the sorter fails, log the first wrong stage and change only the responsible rule, **input**, or label.

When the sorter fails, write competing explanations and predicted **trace**s before changing code or instructions.

Plan evidence, changes and retests

- What real classroom need does this serve?
- What information is necessary—and what should stay private?
- What happens inside the **process**?
- What normal and edge cases will you test?

- When should it say not sure or ask a person?

Keep the changed factor inspectable

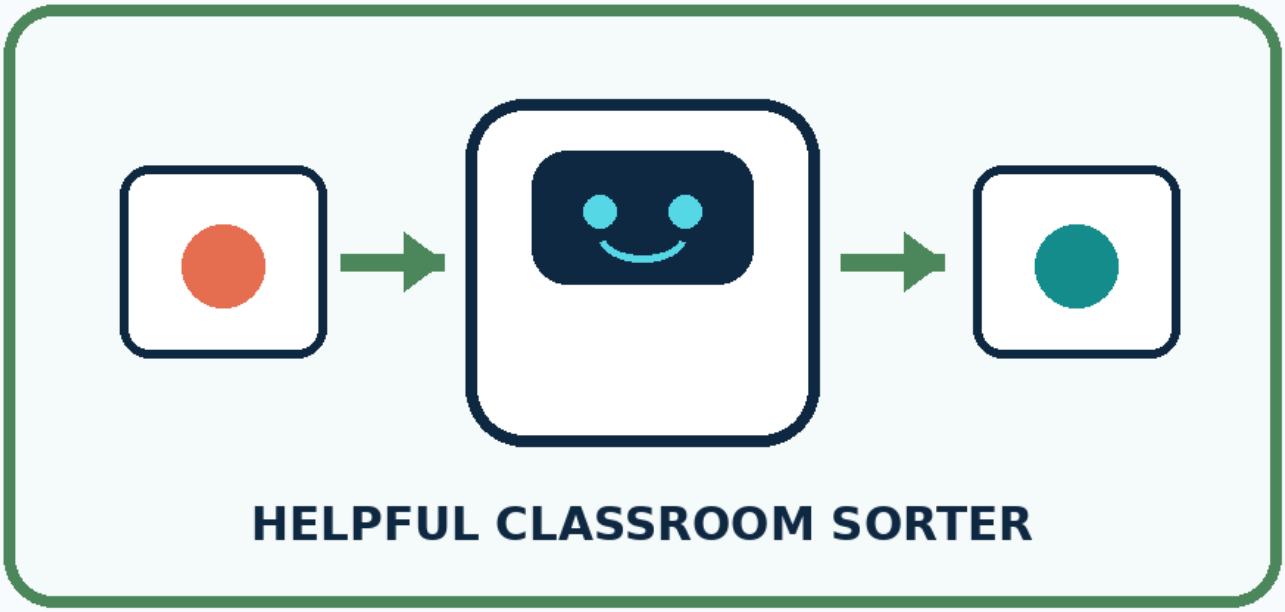
RESPONSIBLE DESIGN A successful new run does not prove the diagnosis if several things changed together.

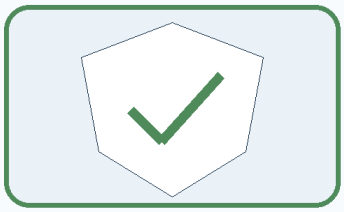
▼ **Earlier wording, alternatives and picture transcripts**

PROJECT

Helpful Classroom Sorter

Build one coherent system across the year, one **evidence**-based decision at a time.





My project decision:

Evidence for your project

MAIN LESSON · SECTION 16

Finish: defend the revealing test

> Close your notes and state the alternatives, their predicted outputs and what an observation could establish.

Your exit explanation

- Close your notes. Explain this goal in your own words: Compare **expected** and observed **trace**s to locate the first difference

> **My deciding step, evidence and one remaining question**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Choose support and revisit later

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	Meaning	My example
bug	cause of incorrect or unexpected behavior	
trace	record of steps and changing state	
expected	result predicted before the test	
actual	result observed during the test	

Terms for the connected explanation

Word	Meaning
Specification	A statement of what a system is required to do
Expected result	The result worked out from the specification before running a test
Observed result	The result actually recorded during a test

Word	Meaning
First divergence	The earliest step where observed behaviour differs from expected behaviour
Regression test	A repeated check that previously correct behaviour still works after a change

Repair a claim using a word

- CLAIM TO REPAIR Debugging means rewriting everything that looks suspicious.
- Use one vocabulary word and one piece of **evidence** to repair it:

▼ Earlier wording, alternatives and picture transcripts

VOCABULARY

Words You Can See and Use

The badge is a memory cue; the definition does the real thinking work.









Misconception detective

WORD POWER A strong definition helps you decide something in a new case.

Words used in the closer explanation

▼ Choose a stronger thinking challenge

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

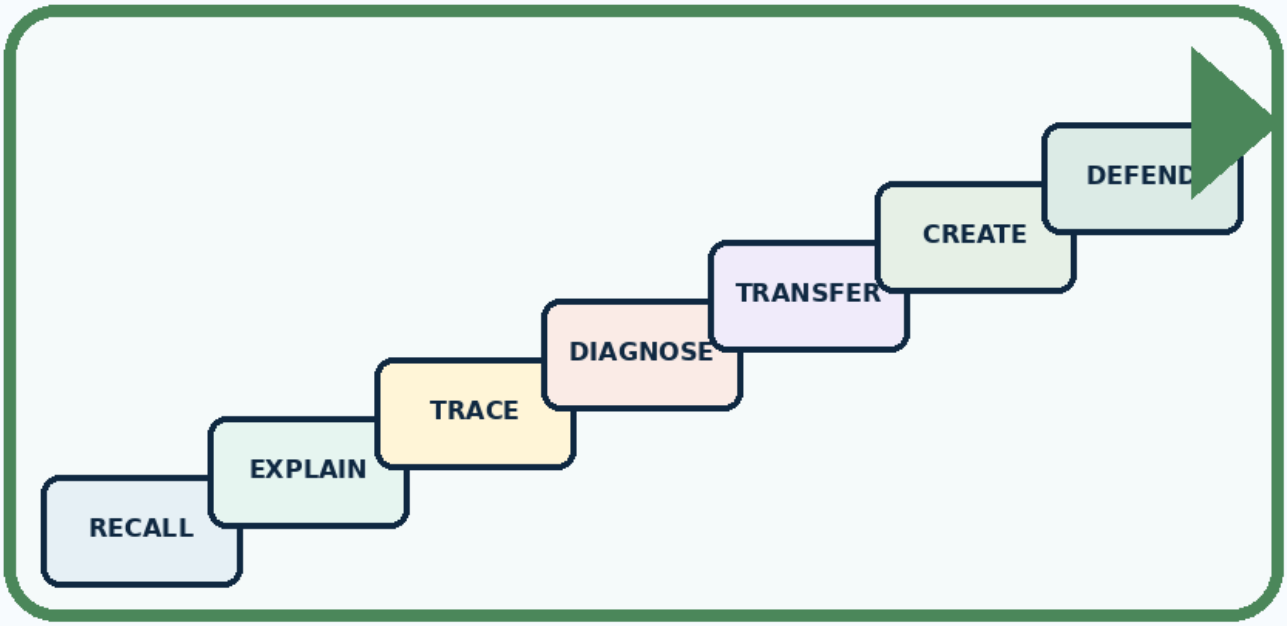
- RECALL Define bug.
- EXPLAIN Debugging finds the broken assumption or step before changing the solution.
- **TRACE** Show the **input**, **process**, and **output**.
- DIAGNOSE Repair: Debugging means rewriting everything that looks suspicious.
- TRANSFER Apply the idea to a new home or classroom case.
- CREATE Design one revealing test.
- DEFEND Give a claim, **evidence**, **mechanism**, limit, and safe action.

▼ Earlier wording, alternatives and picture transcripts

PRACTICE LADDER

Climb from Recall to Defense

Each step asks for a stronger kind of understanding.



RETRY RULE A coached answer becomes mastery only after you solve a different case independently.

▼ **Reference: check your explanation**

REFERENCE · SECTION 12

Reference: check your explanation

> Use after practice. Debug a new ticket rule is the selected independent case.

Check the evidence and its limit

- ☐ Name or draw the fresh case.
- ☐ **Trace input**, **process**, and **output**.

- ☐ State the conclusion and strongest **evidence**.
- ☐ Name one limit, failure, missing clue, or fairness/privacy concern.
- ☐ Explain one safe next action or improvement.

Record support

My level: ☐ Not yet ☐ With one hint ☐ Independently ☐ I can teach it

▼ Earlier wording, alternatives and picture transcripts

MASTERY

Checklist for explaining this week's idea

Use after practice. Debug a new ticket rule is the selected independent case.



EXPLANATION CHECK Keep the specification fixed, locate the first mismatch, predict a distinguishing test and rerun the failure plus earlier passing cases.

Independent **evidence**:

SELF-CHECK **Trace expected** and **actual** positions step by step. The first mismatch reveals the earliest plausible cause.

Change that instruction or assumption only, then rerun the identical maze to show whether the cause was correct.

NEXT BRIDGE Carry today's **evidence** habit into: Learning from Examples.

▼ **Extra practice: Repair the earliest missing action**

REFERENCE · SECTION 13

Extra practice: Repair the earliest missing action

> Try the question before opening its worked explanation.

Read the supplied case and question

A robot should go from square one to square three. Its program is MOVE ONE, STOP.

Find the cause and repair one thing.

Your explanation

- Which detail supports the answer, and why does it matter?

> **Write or draw your reasoning and evidence.**

▼ Check after trying

It stops at square two because one move is missing. Add MOVE ONE before STOP and retest from the same start.

Evidence or stage	Value or result
Intended destination	Square three
Broken program	MOVE, STOP → square two
Repaired program	MOVE, MOVE, STOP → square three

▼ Earlier wording, alternatives and picture transcripts

Week 12 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Repair the earliest missing action

A robot should go from square one to square three. Its program is MOVE ONE, STOP.

Find the cause and repair one thing.

It stops at square two because one move is missing. Add MOVE ONE before STOP and retest from the same start.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Compare expected and observed traces to locate the first difference	Explain the deciding step, show a trace or test, and state one limit.
Choose a test that distinguishes competing explanations of a fault	Explain the deciding step, show a trace or test, and state one limit.
Repair a stated instruction and rerun both the failing case and earlier passing cases	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional paper calculator should double the current value, but a copied instruction behaves strangely.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional paper calculator should double the current value, but a copied instruction behaves strangely.
- Which labelled step matches this detail? This record supports add-6 over doubling for this comparison.
- 1. Observe
- 2. Run revealing test
- 3. Predict alternatives
- 4. Locate cause
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Locate cause This record supports add-6 over doubling for this comparison. Repeating **input** 6 would not separate the two hypotheses because both again predict 12.

Input 2 makes their **prediction**s different before the test is run. The second record supports investigating an added constant, but inspecting the instruction is still needed to confirm the implementation.

Changed-case question

- Changed-case practice: Use **input** 6 again instead of 2. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. Both hypotheses predict 12; the test remains inconclusive.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial **trace**. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed case**

After your attempt, compare with the supported explanation: Both hypotheses predict 12; the test remains inconclusive. A revealing test requires different predicted **output**s, not merely another recorded run.

Repair and save your work

Review Repeating Efficiently (ai-1/11). Goal: **Trace** a loop by testing its condition before every repetition. Short practice: **Trace** a fresh start of 9. How many additions occur, and must the report equal 10?.

Return to this week and explain what you changed.

Review Algorithms as Recipes (ai-1/9). Goal: **Trace** an ordered algorithm using the current value after every instruction. Short practice: Use the arrows to explain where the two zero-start runs first differ..

Return to this week and explain what you changed.

Review Sound Becomes a Signal (ai-1/6). Goal: Interpret an ordered sequence of sound-signal samples with a stated time interval. Short practice: Does the all-zero retained record prove the full signal was silent?

Use the picture.. Return to this week and explain what you changed.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

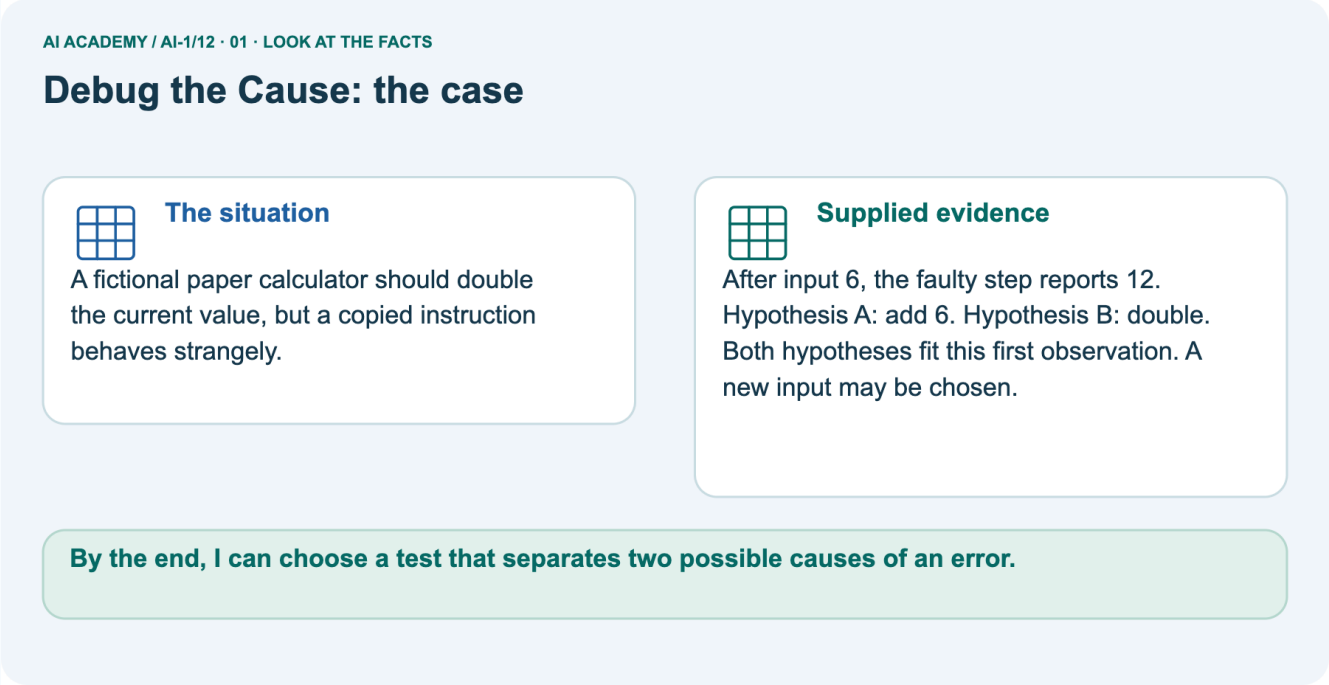
Keep your completed work

▼ Reference: the six-picture journey

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Debug the Cause: the case



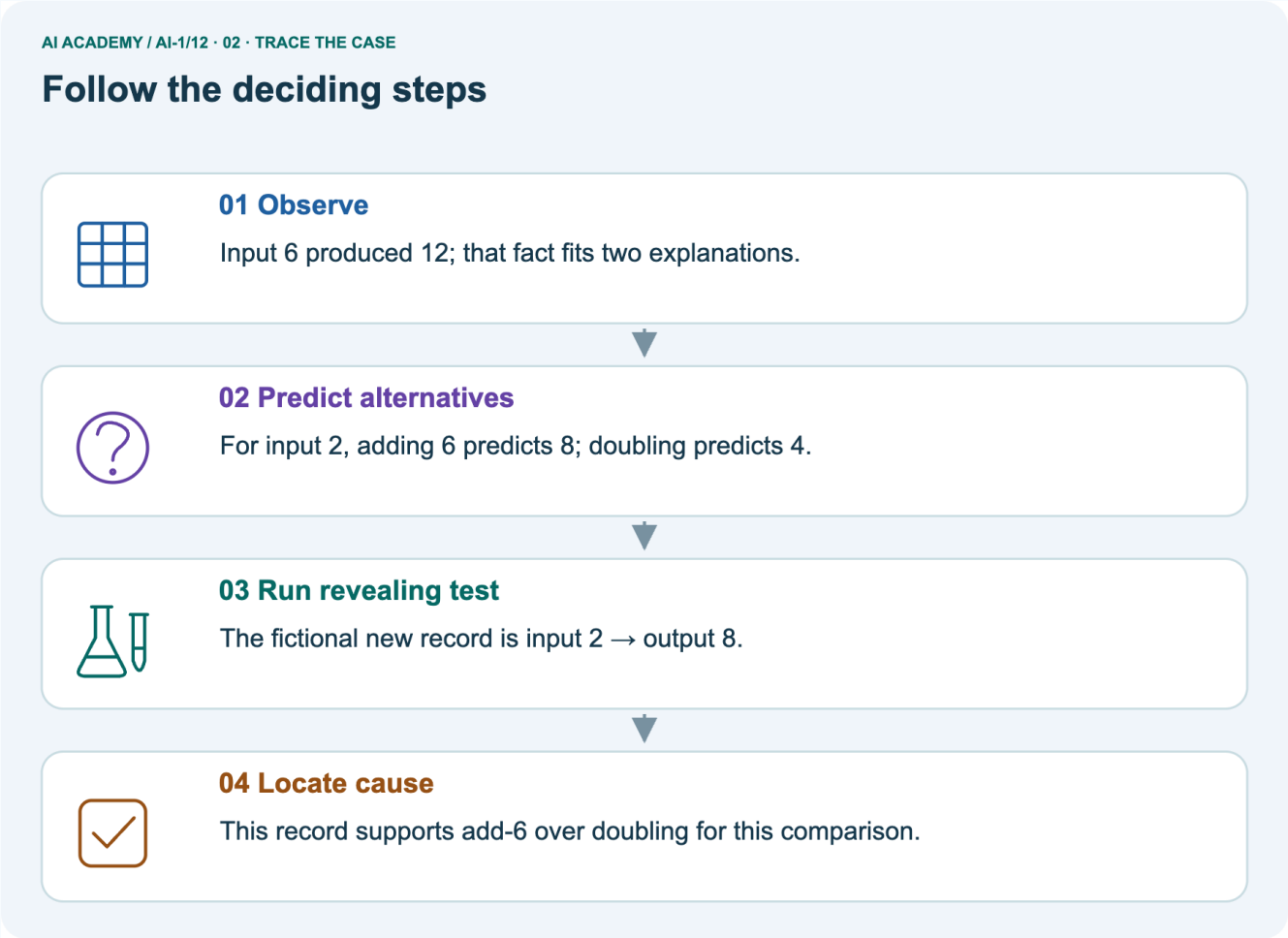
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional paper calculator should double the current value, but a copied instruction behaves strangely. Supplied facts: After **input** 6, the faulty step reports 12. Hypothesis A: add 6. Hypothesis B: double.

Both hypotheses fit this first **observation**. A new **input** may be chosen.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

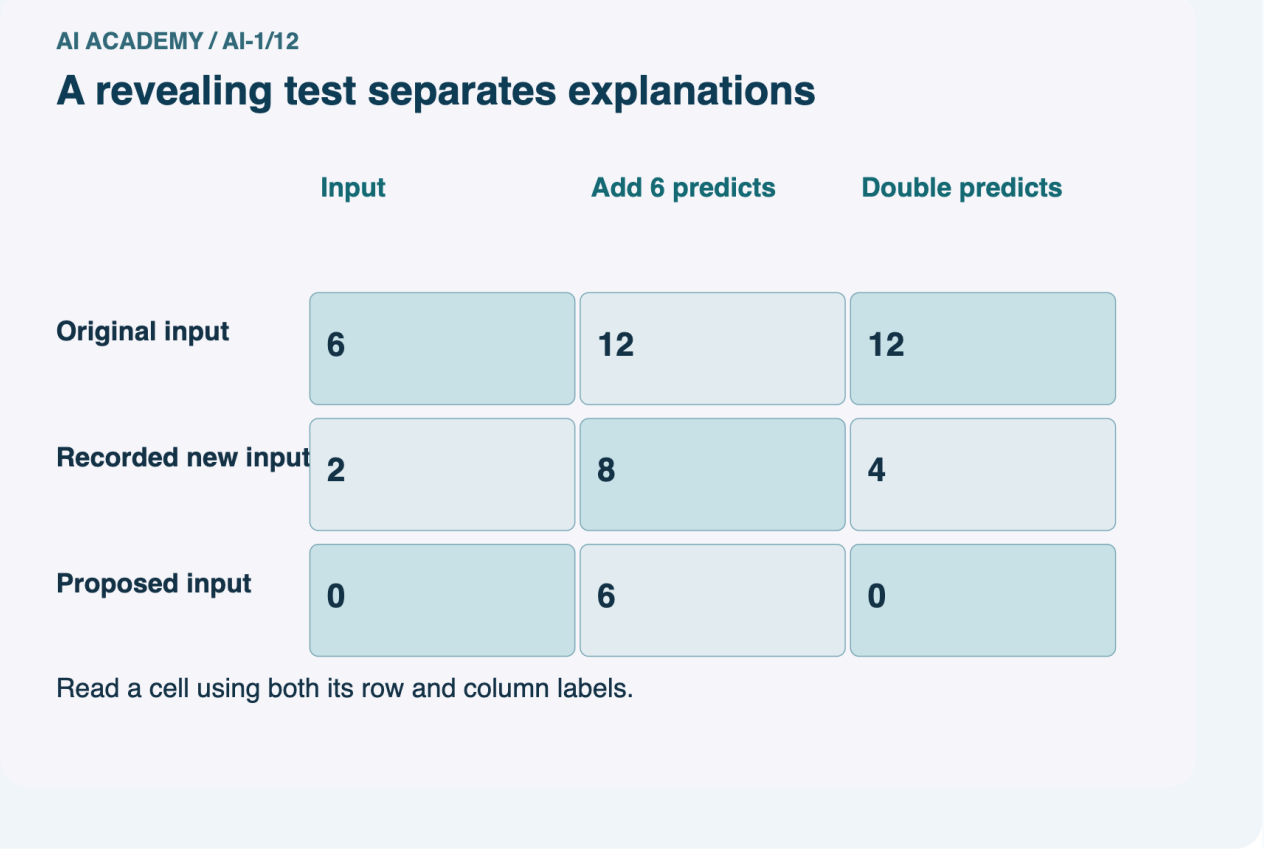


Follow the labelled picture with the matching explanation. Open the vector diagram

Observe: **Input** 6 produced 12; that fact fits two explanations. → Predict alternatives: For **input** 2, adding 6 predicts 8; doubling predicts 4. → Run revealing test: The fictional new record is **input** 2 → **output** 8. → Locate cause: This record supports add-6 over doubling for this comparison.

Point to each numbered stage and explain which recorded **input** it uses.

03. A revealing test separates explanations



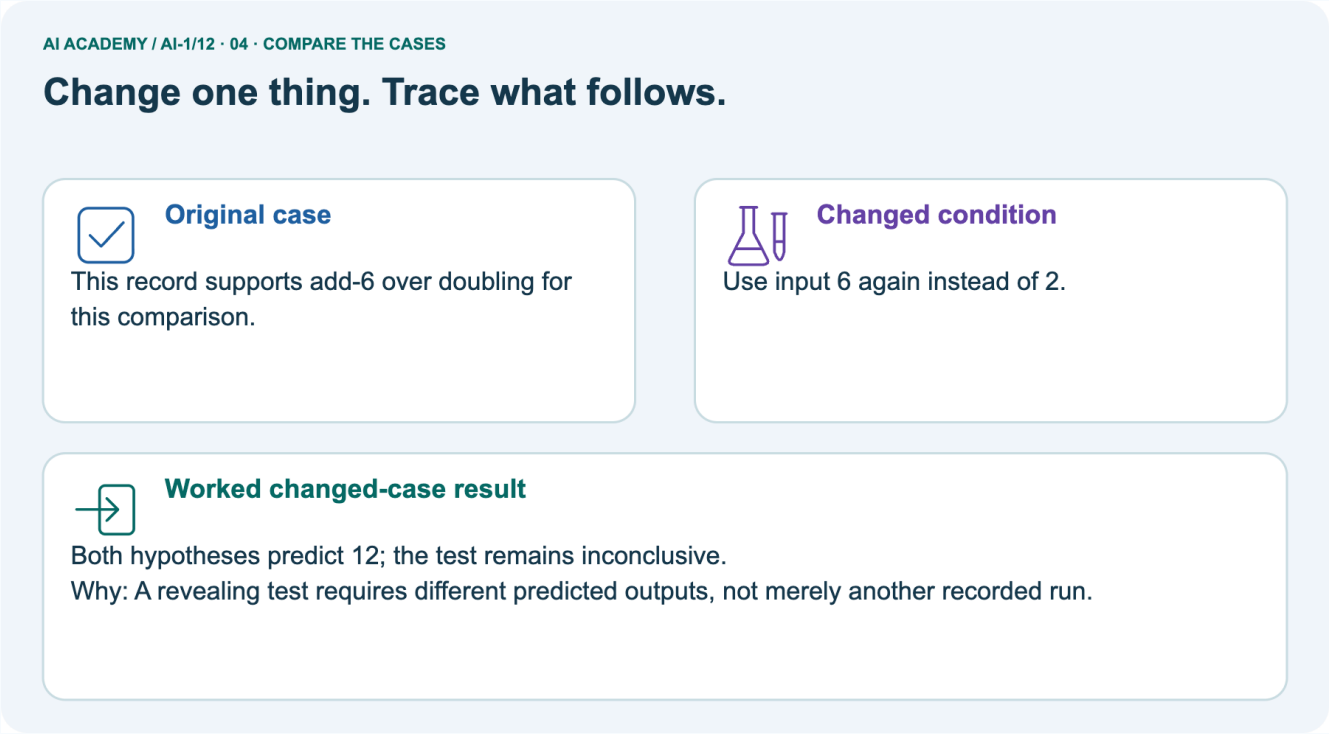
Follow the labelled picture with the matching explanation. Open the vector diagram

A revealing test separates explanations. Original **input**: **Input** = 6, Add 6 predicts = 12, Double predicts = 12. Recorded new **input**: **Input** = 2, Add 6 predicts = 8, Double predicts = 4.

Proposed **input**: **Input** = 0, Add 6 predicts = 6, Double predicts = 0.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



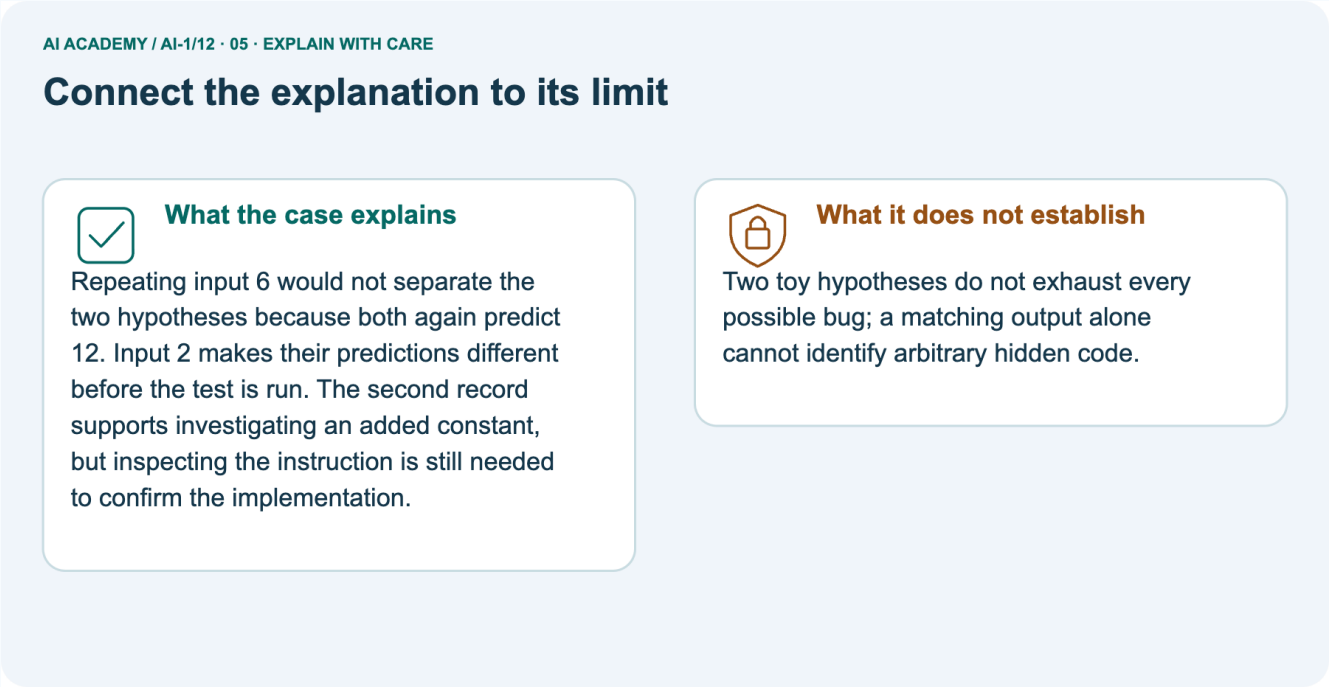
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: This record supports add-6 over doubling for this comparison. Changed condition: Use **input** 6 again instead of 2. **Expected result**: Both hypotheses predict 12; the test remains inconclusive.

Reason: A revealing test requires different predicted **output**s, not merely another recorded run.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

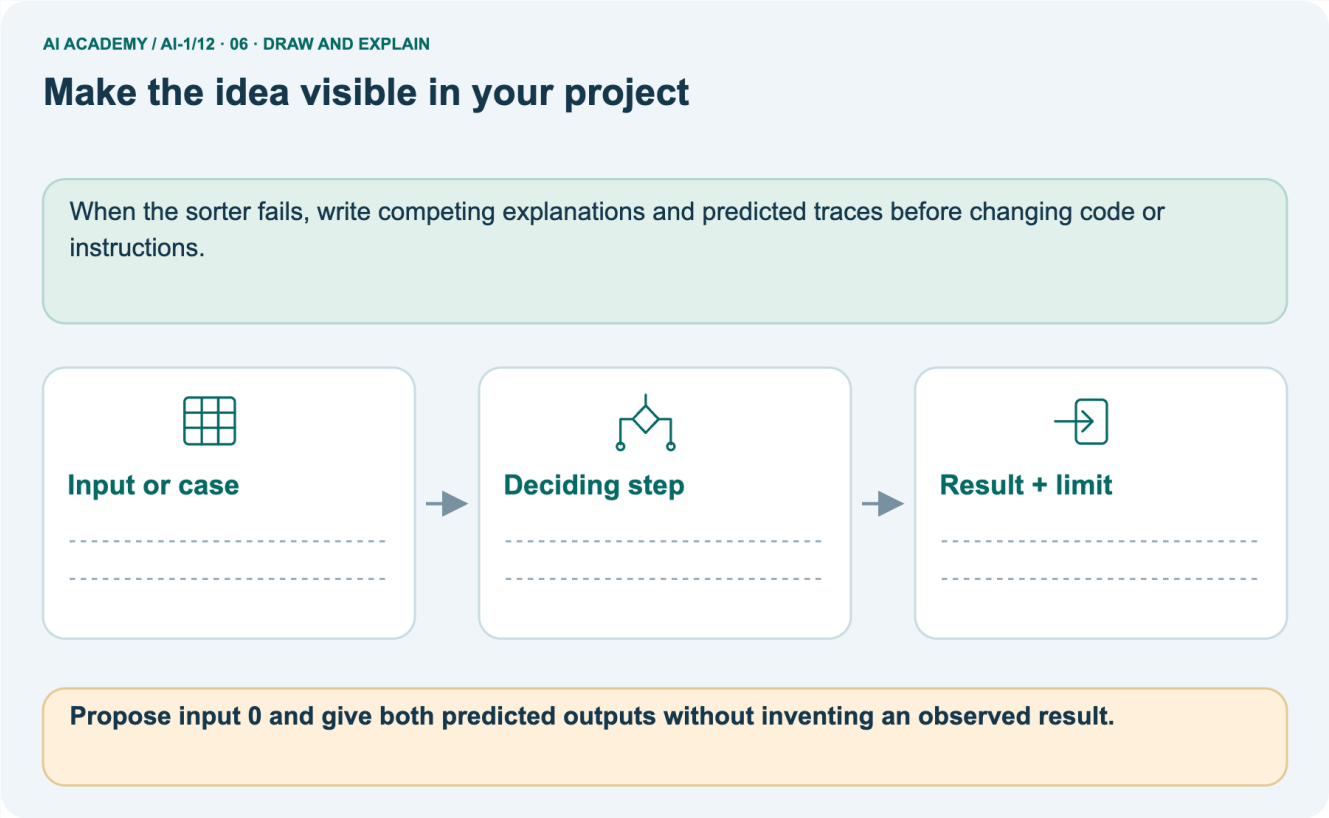
Repeating **input** 6 would not separate the two hypotheses because both again predict 12. **Input** 2 makes their **prediction**s different before the test is run.

The second record supports investigating an added constant, but inspecting the instruction is still needed to confirm the implementation.

Limit: Two toy hypotheses do not exhaust every possible bug; a matching **output** alone cannot identify arbitrary hidden code.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

When the sorter fails, write competing explanations and predicted **trace**s before changing code or instructions. Draw your **input**, deciding step, result and limit.

Supported question: Propose **input** 0 and give both predicted **output**s without inventing an **observed result**.

Draw your project **trace** and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Debug the Cause: the case

02. Follow the deciding steps

03. A revealing test separates explanations

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.